北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Tang	**名 First Name**	Ziming		
BUPT 学号 BUPT number	2022213090	**QM 学号 QM number**	221165430	**班级 Class**	2022215103
论文题目 Project Title	Tracking progress of Final year projects for students				

是否完成任务书中所定的中期目标？Targets met (as set in the Specification)?

No, due to medical conditions I was taking treatment with mild depression. Related justification can be given.

已完成工作 Finished work:

1 Core User Authentication and Interface Design

1) Working Login/Registration Module: The complete and secure

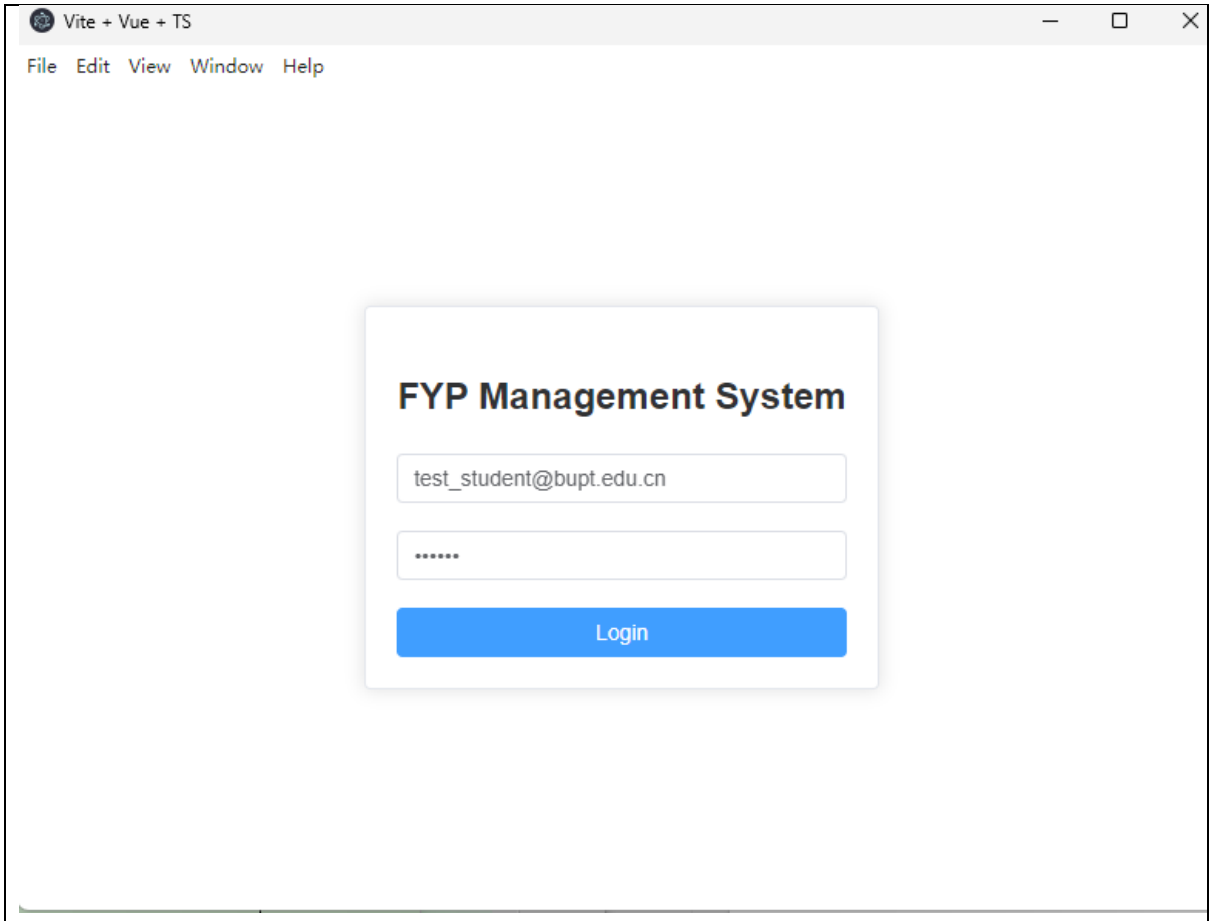
authentication system must be functional, allowing both Staff and Student roles to

successfully login and access their respective main interfaces.

2) User-Friendly Interfaces: The basic design and navigation for both the

supervisor and student interfaces must be complete and demonstrate a high

degree of usability



Above is the login user interface, the system can automatically distinguish students and supervisors and direct them to different pages respectively.

```
@RestController   Tang Ziming *
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private UserService userService;

    @PostMapping("/register")   Tang Ziming
    public Result<String> register(@RequestBody User user) { return Result.success(userService.reg

    @PostMapping("/login")   Tang Ziming *
    public Result<String> login(@RequestBody LoginDataDTO loginData) {
        String token = userService.login(loginData.getEmail(), loginData.getPassword());
        return Result.success(token);
    }

    @PostMapping("/update-password")   Tang Ziming *
    public Result<String> updatePassword(@RequestBody PasswordUpdateDTO dto){
        UserContext.UserDTO userDTO = UserContext.getUser();
        userService.updatePassword(userDTO.getEmail(), dto.getOldPassword(), dto.getNewPassword())
        return Result.success( data: "Update Success");
    }
}
```

Above is the related backend controller in Java Springboot. It allows users to register, login and change their password. It serves as the first layer of the 3-layer architecture backend system. Using Service layer to complete business logic. In Service layer, it calls the database and extract, modify the data of user database e.g., login method provides token for user to hold the access ability to use the functions of the system.

```java
public class JwtInterceptor implements HandlerInterceptor {
    private JwtUtils jwtUtils;

    @Override  Tang Ziming *
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object har
        if ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
            return true;
        }

        String token = request.getHeader( name: "Authorization");

        if (token != null && token.startsWith("Bearer ")) {
            token = token.substring( beginIndex: 7);

            // 解析即验证
            Claims claims = jwtUtils.getClaimsByToken(token);

            if (claims != null) {
                // 验证通过，封装用户信息
                UserContext.UserDTO userDTO = UserContext.UserDTO.builder()
                        .userId((Integer) claims.get("userId"))
                        .email(claims.getSubject())
                        .role((String) claims.get("role"))
                        .build();

                UserContext.setUser(userDTO);
                return true;
            }
        }

        // 验证失败：Token 缺失、过期、签名错误都会走到这里
        response.setContentType("application/json;charset=UTF-8");
        response.setStatus(401);
        response.getWriter().write( s: "{\"code\": 401, \"msg\": \"Unauthorized: Invalid or expire
        return false;
```

Above is the interceptor which is used as a filter of the backend system. It can extract the token from frontend i.e., the indicator that specifies a user that logins, and verifies the correctness of the related user, based on which the interceptor lets the user pass or denies.

JWT is a techniques that stores 3 different kinds of information: algorithm, claims and signature. Different algorithm can use different method encrypt the token, claims restore messages specified by the user and signature is the indicator that server specifies.

2 Supervisor Core Tracking Capability

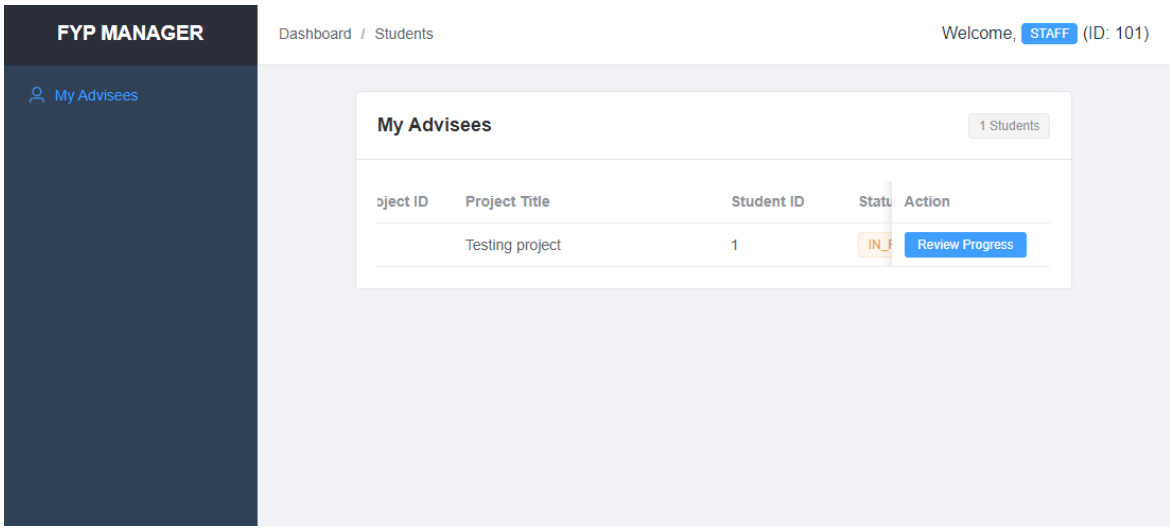
1) Supervisor Dashboard (Total Students Page): The supervisor must be able

to successfully login and view a page listing all their supervised students.

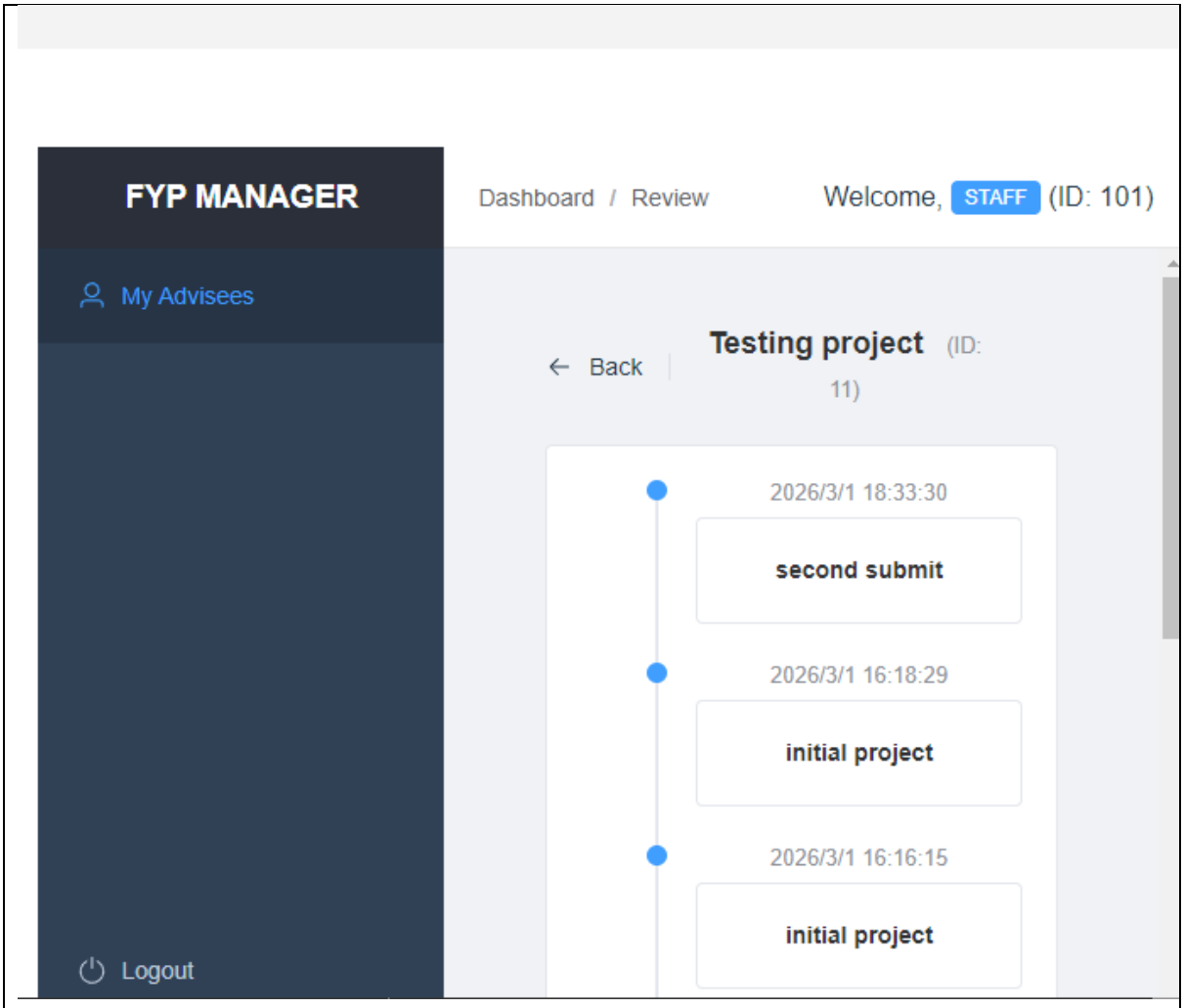
2) Individual Student Progress View: The supervisor must be able to navigate

to an individual student's page and view a basic version chain/log of the student's

work or tasks, demonstrating the fundamental data retrieval mechanism.



Above is the dashboard supervisors can see after login. It shows the students the supervisor manages and their respective projects, i.e., project ID, title, student ID, status and them can click to see the progress of the project



Above is for supervisor check the student's project progress. It shows when the commits takes place and the description of the commit. Also the staff ID is displayed.

```java
public class SupervisorProjectController {
    @GetMapping("/my-students")
    public Result<List<Project>> getMyStudents() {
        Integer supervisorId = UserContext.getUserId();
        // 强制只查当前导师 ID 关联的所有项目
        List<Project> list = projectService.list(new LambdaQueryWrapper<Project>()
                .eq(Project::getSupervisorId, supervisorId));
        return Result.success(list);
    }

    /**
     * 2. 查看单个学生的项目进展记录 (Individual Student Progress View)
     * 逻辑：根据项目ID获取该项目的所有版本日志，按时间倒序排列
     */
    @GetMapping("/{projectId}/progress")
    public Result<List<VersionLog>> getStudentProgress(@PathVariable Integer projectId) {
        // 安全校验：确保该项目确实属于当前登录的导师
        Project project = projectService.getById(projectId);
        if (project == null || !project.getSupervisorId().equals(UserContext.getUserId())) {
            return Result.error( code: 403,  msg: "Access denied.");
        }

        // 获取该项目的所有日志，按时间从新到旧排序
        List<VersionLog> logs = versionLogService.list(new LambdaQueryWrapper<VersionLog>()
                .eq(VersionLog::getProjectId, projectId)
                .orderByDesc(VersionLog::getCreatedAt));

        return Result.success(logs);
    }
}
```

Above is the SupervisorProjectController which provides api interfaces for frontend logic. Like AuthController, it provides basic data extraction from database and list all the information that frontend needs.

3 Student Core Interaction Functionality

1) File/Document Upload Module: The student must be able to successfully

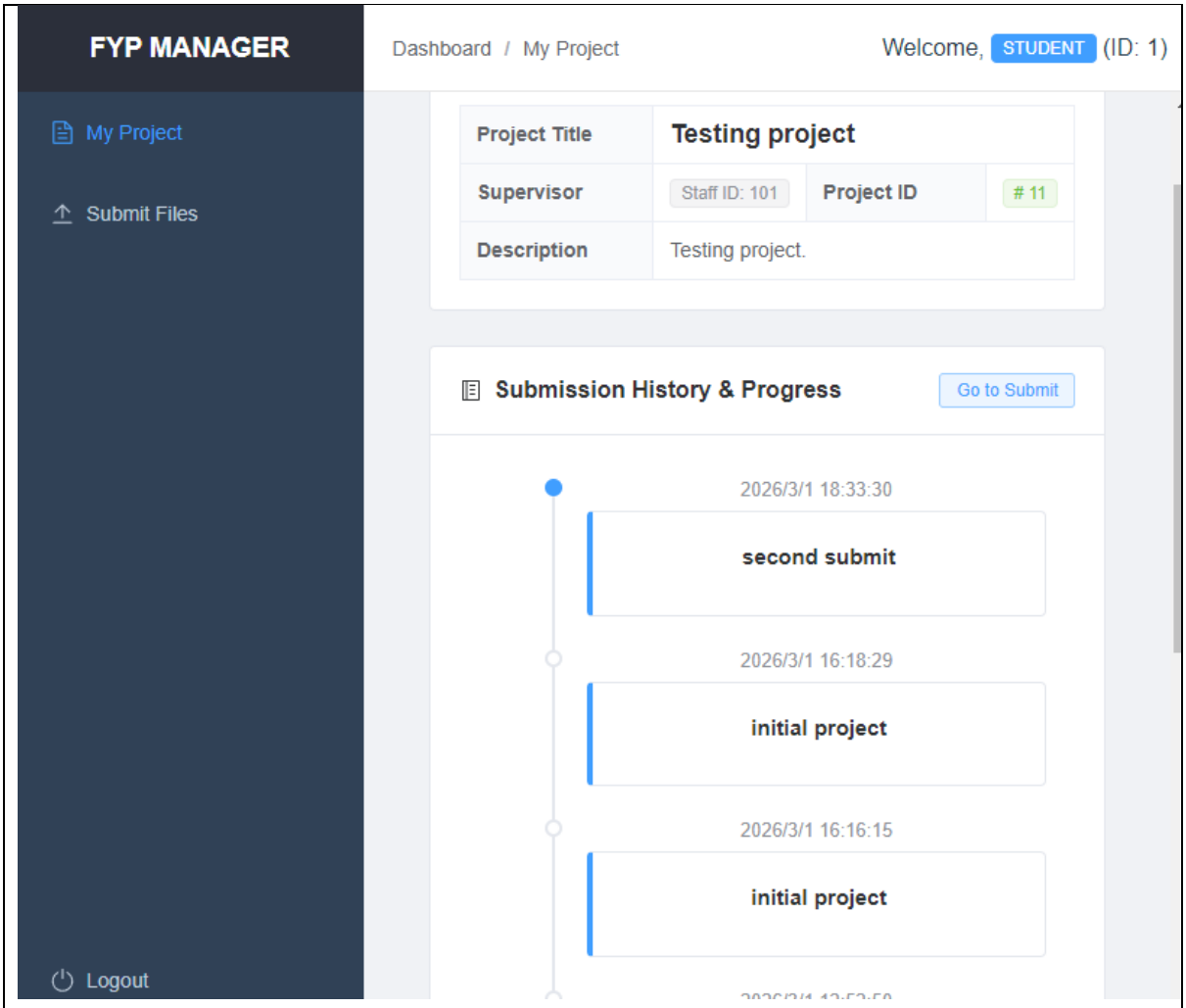
login and use a functional interface to upload a document or project file to the

system's storage (OSS API or simulated storage).

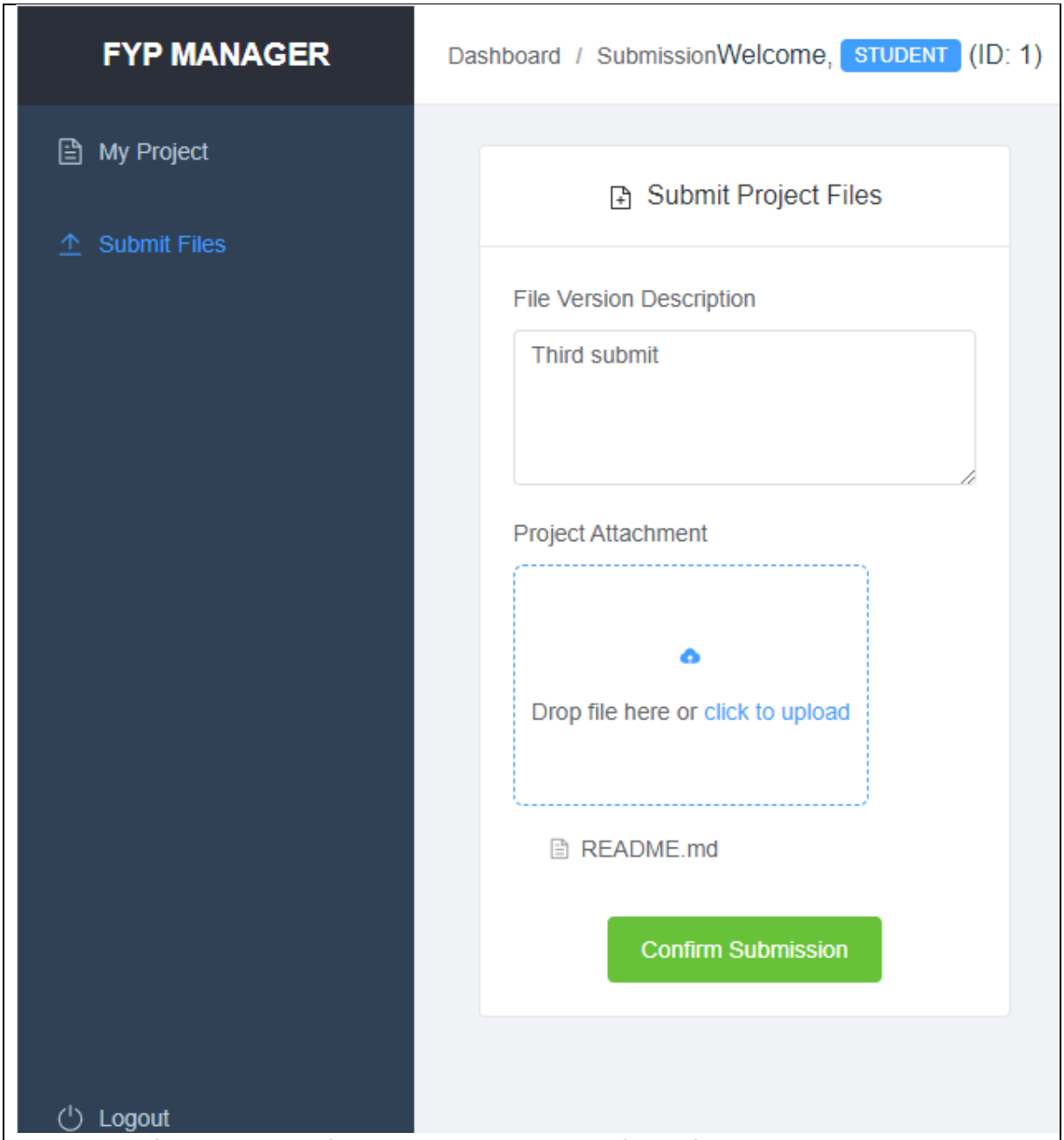
2) Basic Version Control Link View: The student must be able to view their

own uploaded files/links and the associated version chain/history log within their

dedicated interface.



Above is the page that students can see after login. It is the dashboard of students' project and project ID, title supervisor and description can be seen. Also the project progress is displayed and can see the related commit time and description.

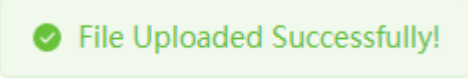


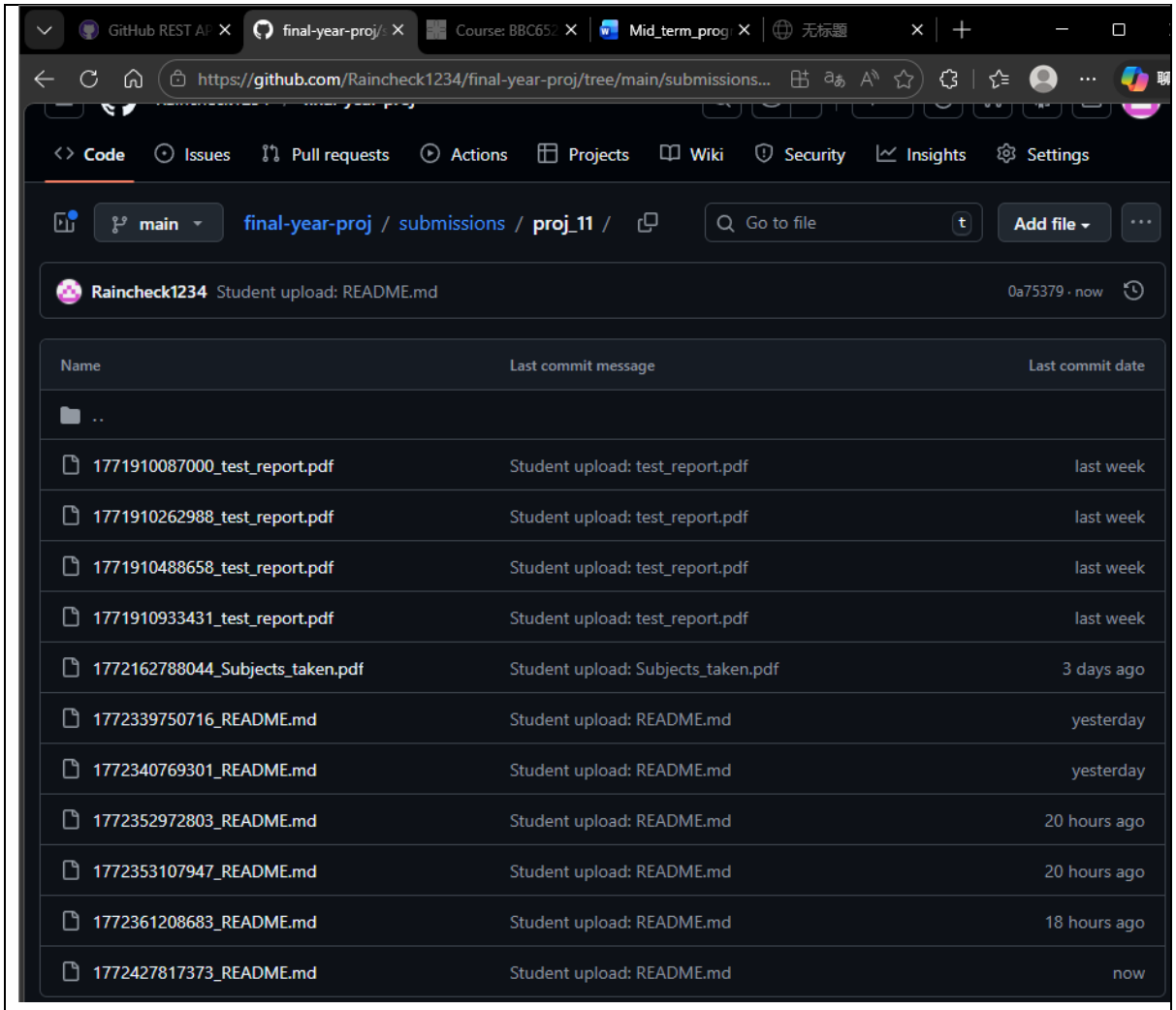
Above is the file upload page for students to upload their files. After type in description and the project attachment, click the confirm submission button and the corresponding files will be sent to the final-year-proj github repo.

```java
22  public class StudentProjectController {
35      public Result<ProjectVO> getMyProject() {
37
38          // 使用你自定义的 Service 方法进行联表查询
39          // 如果还没写联表，可以先用这种简单方式凑效：
40          Project project = projectService.getOne(new LambdaQueryWrapper<Project>()
41                  .eq(Project::getStudentId, studentId));
42
43          if (project == null) return Result.error( code: 404,  msg: "No project found");
44
45          ProjectVO vo = new ProjectVO();
46          BeanUtils.copyProperties(project, vo);
47
48          // 假设你有 StaffService，根据 staffId 查名字
49          if (project.getSupervisorId() != null) {
50              User staff = userService.getById(project.getSupervisorId());
51              vo.setSupervisorName(staff != null ? staff.getName() : "Not Assigned");
52          }
53
54          return Result.success(vo);
55      }
56
57      // B. 学生提交作业 (已有的)
58      @PostMapping("/{projectId}/upload")  新 *
59      public Result<UploadedFile> upload(@PathVariable Integer projectId,
60                                         @RequestPart MultipartFile file,
61                                         @RequestParam(required = false) String description) {
62          Project project = projectService.getById(projectId);
63
64          // 修复点：确保类型一致。建议统一使用 Long 或在比较时转为 String
65          if (project == null || !String.valueOf(project.getStudentId()).equals(String.valueOf(UserC
66              return Result.error( code: 403,  msg: "Access Denied. Project ID: " + projectId + " does
67          }
68
69          // 如果这里报错，说明 uploadSubmission 内部逻辑有问题
70          return Result.success(projectService.uploadSubmission(projectId, file, description));
```

```java
public class StudentProjectController {
    public Result<UploadedFile> upload(@PathVariable Integer projectId,
    }

    /**
     * C. 满足需求 3.3 (2): 查看自己的版本记录/进展日志
     * 学生需要看自己什么时候交了什么，以及对应的 GitHub 链接
     */
    @GetMapping("/{projectId}/progress")
    public Result<List<VersionLog>> getMyProgress(@PathVariable Integer projectId) {
        // 安全校验：防止学生 A 查学生 B 的进度
        Project project = projectService.getById(projectId);
        if (project == null || !project.getStudentId().equals(UserContext.getUserId())) {
            return Result.error(403, "Access Denied.");
        }

        // 查询该项目下所有的日志，按时间倒序排列
        List<VersionLog> logs = versionLogService.list(new LambdaQueryWrapper<VersionLog>()
                .eq(VersionLog::getProjectId, projectId)
                .orderByDesc(VersionLog::getCreatedAt));

        return Result.success(logs);
    }

    // D. 学生创建项目 (已有的)
    @PostMapping("/create")
    public Result<String> createProject(@RequestBody Project project) {
        // 修正：强制设置 studentId 为当前登录者，防止前端传参欺骗
        project.setStudentId(UserContext.getUserId());
        projectService.createProject(project);
        return Result.success("Project application submitted.");
    }
}
```

Above is the StudentProjectController which provides api interfaces to not only show project progress but also upload a file. Not like backend logic before, it integrates github RESTAPI to upload the commits to the corresponding github repository.





We choose to use one guthub repository to store all the student files and progress. The files can be seen in the github repository as above pic shows. Thanks to github RESTAPI we can upload files in the remote repository.

4 Corresponding database design

```
CREATE TABLE users (
        user_id INT AUTO_INCREMENT PRIMARY KEY,
        role ENUM('STUDENT','STAFF') NOT NULL,
        name VARCHAR(100) NOT NULL,
        email VARCHAR(150) NOT NULL UNIQUE,
        password_hash VARCHAR(255) NOT NULL,
        created_at DATETIME NOT NULL,
        last_login DATETIME
);

CREATE TABLE projects (
        project_id INT AUTO_INCREMENT PRIMARY KEY,
        student_id INT NOT NULL,
        supervisor_id INT NOT NULL,
        project_title VARCHAR(255) NOT NULL,
        description TEXT,
        created_at DATETIME NOT NULL,
        updated_at DATETIME
```

```
);

CREATE TABLE version_logs (
                    log_id INT AUTO_INCREMENT PRIMARY KEY,
                    project_id INT NOT NULL,
                    log_type ENUM('COMMIT','UPLOAD','NOTE') NOT NULL,
                    commit_link VARCHAR(255),
                    description TEXT,
                    timestamp DATETIME NOT NULL,
                              created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE uploaded_files (
                     file_id INT AUTO_INCREMENT PRIMARY KEY,
                     project_id INT NOT NULL,
                     file_name VARCHAR(255) NOT NULL,
                     file_url VARCHAR(255) NOT NULL,
                     size_mb FLOAT,
                     uploaded_at DATETIME NOT NULL
);

CREATE TABLE blog_posts (
                   post_id INT AUTO_INCREMENT PRIMARY KEY,
                   author_id INT NOT NULL,
                   project_id INT NOT NULL,
                   content TEXT NOT NULL,
                   created_at DATETIME NOT NULL
);

CREATE TABLE blog_interactions (
                       interaction_id INT AUTO_INCREMENT PRIMARY KEY,
                       post_id INT NOT NULL,
                       user_id INT NOT NULL,
                       type ENUM('COMMENT','LIKE','SEEN') NOT NULL,
                       content TEXT,
                       timestamp DATETIME NOT NULL
);
```

Above is the databases designed to store all the information needed in the final year project system. The foreign keys are deployed outside the database but in the backend system.

尚需完成的任务 Work to do:

3.4 External Communication Integration

MS Teams API Integration (Basic Chat): A basic, functional integration with the

MS Teams API must be implemented, allowing either the Staff or Student user to

initiate a chat or call via a linked interface component.

存在问题 Problems:

1. Some bugs still exist which prevent clicks on project title jump to the link of the submission in the github repository.

2. The UI interface still has layout problem that needs to be fixed.

拟采取的办法 Solutions:

1. Learn how to integrate Teams interfaces and use it to strengthen the system
2. Fix the bugs and let the title direct to the corresponding commit links
3. Fix the UI interface problem

论文结构 Structure of the final report: (Chapter headings and section sub headings)

Chapter 1: Introduction

1.1 Project Overview

1.2 Motivation

1.3 Aims and Objectives

1.4 Scope and Constraints

1.5 Report Structure

Chapter 2: Background

2.1 Technical Context

2.2 Related Work / Literature Review

2.3 Problem Statement

2.4 Novelty / Contribution

Chapter 3: Design and Implementation

3.1 System Architecture

3.2 Requirements Analysis

3.3 Data Model / Database Design

3.4 Key Component Design

3.5 Implementation Details

Chapter 4: Results and Discussion

4.1 Evaluation Methodology

4.2 Functional Testing

4.3 Performance Analysis

4.4 User Feedback / Acceptance Testing

4.5 Discussion of Findings